

Option 2: with AMQP

Step 1: The PWA Extraction (Person 2)

- **What happens:** The user records the set, and MediaPipe extracts the coordinates locally on the phone. Person 2 sends the JSON payload to the .NET Core API.
- **The Contract (JSON Payload 1):** `POST /api/vision/analyze`
- JSON

```
{
  "userId": "user_789",
  "exercise": "squat",
  "view": "front",
  "frames": [
    { "timestamp": 0.0, "landmarks": [{"x": 0.5, "y": 0.6, "z": 0.1}, ...] },
    { "timestamp": 0.3, "landmarks": [...] }
  ]
}
```

-
-

Step 2: The Service Bus "Push" (Person 3 & Person 4)

- **What happens:** The Core API saves the heavy JSON to Azure Blob Storage (because Service Bus has a 256KB message limit).
- Instead of writing to a slow Storage Queue, the Core API publishes a message to the Azure Service Bus Queue.
- Because Person 4's Python script has an open AMQP connection to Azure, Azure instantly pushes the message down the tunnel to the Home PC with zero delay.
- **The Contract (Service Bus Message Payload):**
- JSON

```
{
  "jobId": "job_12345",
  "exercise": "squat",
  "view": "side",
  "blobUrl": "https://saoptilifts.blob.core.windows.net/exercises/job_12345.json"
```

```
}
```

-
-

(Note for Person 4: In Python, you won't use a polling loop anymore. You will use the `azure-servicebus` package, which looks like this:)

Python

```
# The AMQP connection stays open automatically.
```

```
# This function fires instantly the millisecond Azure pushes a job.
```

```
with servicebus_receiver:
```

```
    for msg in servicebus_receiver:
```

```
        process_job(msg.body)
```

```
        servicebus_receiver.complete_message(msg) # Fault tolerance!
```

Step 3: Distributed Execution (Person 4 & Person 1)

- **What happens:** The Python worker downloads the JSON from the `blobUrl`, passes it to Person 1's PyTorch/Scikit-Learn model, and detects the errors.
- Once finished, the Worker sends a standard HTTP POST back to the .NET Core API to hand in the results.
- **The Contract (JSON Payload 2):** `POST /api/vision/worker-result`
- JSON

```
{
```

```
  "jobId": "job_12345",
```

```
  "success": true,
```

```
  "detected_anomalies": ["excessive_forward_lean"]
```

```
}
```

Step 4: The LLM Layer (Person 3)

- **What happens:** The .NET Core API receives Payload 2. Person 3 intercepts the `detected_anomalies` array and formats a prompt for the Gemini 1.5 Flash API.
- **Prompt:** *"User squat errors: excessive_forward_lean. Write a 2-sentence tip."*
- Gemini returns the coaching string. Person 3 saves it to PostgreSQL and marks the job as Completed.

Step 5: The Final UI (Person 2)

- **What happens:** While all of this was happening, the PWA was showing a "Processing..." screen, hitting `GET /api/vision/result/{jobId}` every 2 seconds.
- Once Step 4 finishes, the API finally returns a `200 OK` with the final data.
- **The Contract (JSON Payload 3):**
- JSON

```
{
  "status": "completed",

  "coach_summary": "Great effort! I noticed your chest dropping too far forward on the way up.
Try to keep your chest proud and push through your heels to stay upright."
}
```

- The PWA displays the summary to the user.

Why this is the ultimate architecture:

1. **Zero Latency:** The handoff from Azure to the Home PC takes milliseconds.
2. **Bulletproof Firewall:** Your Home PC makes an outbound request to Azure Service Bus. The ISP leaves the port open, requiring zero configuration.
3. **Fault Tolerance:** If your Home PC crashes in the middle of a job, it fails to send the `complete_message(msg)` signal. Service Bus will automatically put the job back in the queue so it isn't lost.